1. **To understand the basic concepts of version control and perform fundamental Git operations such as repository creation, staging, committing, and viewing commit history.**

Objective:

To introduce students to the principles of Distributed Version Control Systems (DVCS) using Git. This module provides hands-on experience in initializing repositories, managing file lifecycles across the working directory, staging area, and local repository, and inspecting historical project snapshots.

Learning Outcomes

By the end of this lab, students will be able to:

- **Distinguish** between local, staged, and committed file states within the Git workflow.
- **Install and configure** Git on Linux, macOS, or Windows environments.
- **Initialize** a new Git repository (git init) and verify repository tracking status (git status).
- **Stage and commit** project changes using git add and git commit with meaningful commit messages.
- **Inspect** repository history using git log with various formatting flags (--oneline, --stat, --graph).

Procedure:

Installing Git

Linux (Ubuntu / Debian)

sudo apt update

sudo apt install git -y

Linux (Fedora / RHEL)

sudo dnf install git -y

macOS

Install via **Homebrew** or using macOS developer tools:

```
# Using Homebrew

brew install git


# Alternatively, trigger the Xcode command line tool installer

git --version
```

Windows

1. Download the official installer from git-scm.com.
2. Run the executable and follow the setup wizard.
3. Keep default options selected (ensuring **Git Bash** and system PATH integration are checked).

2. Initial One-Time Configuration

Before creating commits, configure your user identity and default branch name globally across your system:

```
# Set your name and email (included in every commit metadata)

git config --global user.name "Your Name"

git config --global user.email "your.email@example.com"


# Set the default primary branch name to 'main'

git config --global init.defaultBranch main


# Verify configuration settings

git config --list
```

Hands-On Lab Procedure

1.1. Repository Initialization: Workspace Setup.

Create a project folder and transform it into a tracked Git repository.

```
# Create a project directory and enter it
mkdir ~/git_demo
cd ~/git_demo

# Initialize an empty Git repository (.git directory created)
git init
```

2.2. Creating and Inspecting Untracked Files: File State Tracking.

Create sample project files and inspect how Git detects untracked changes.

```
# Create sample files
echo "# Git Fundamentals Demo" > README.md
echo "console.log('Hello Git');" > app.js

# Check repository status (Files listed under 'Untracked files')
git status
```

3.3. Staging Changes for Commit: Staging Area.

Move files from the working directory into the Git Staging Area (Index).

```
# Stage a single file
git add README.md

# Stage all remaining files in the current directory
```

```
git add .


# Verify staged files (Listed under 'Changes to be committed')
git status
```

4.4. Committing Staged Changes: Creating Snapshots.

Record a permanent snapshot of the staged files into the local repository history.

```
# Create a commit with a clear, concise descriptive message
git commit -m "Initial commit: Add README.md and app.js"


# Verify that working directory is clean
git status
```

5.5. Modifying Files and Reviewing Differences: Iterative Workflow.

Update existing files, inspect exact modifications before staging, and create a second commit.

```
# Append content to README.md
echo "This lab covers basic Git operations." >> README.md


# Compare changes between working directory and the last commit
git diff


# Stage and commit the updated file
git add README.md
git commit -m "docs: Update README with lab summary"
```

6.6. Viewing Commit History: History Inspection.

Inspect the linear chain of snapshots recorded in the repository.

Display full history details (Commit Hash, Author, Date, Message)

git log

Display a condensed one-line summary per commit

git log --oneline

Display history with change statistics per commit

git log --stat

complete guide for:

Command	**Description**
git init	Initializes a new empty Git repository in the current directory
git status	Displays the status of working directory and staging area files
git add <file>	Stages specified file changes for the next commit snapshot
git add .	Stages all modified and untracked files in the current working directory
git commit -m "msg"	Permanently saves staged changes into repository history

Command	Description
git diff	Shows exact line-by-line differences between working copy and staging area
git log	Displays chronological commit history log
git log --oneline	Displays commit history in a simplified single-line format